

Day6 实训手册：RabbitMQ — 异步与削峰填谷

一、今日任务目标

目标	说明
掌握 MQ 核心概念	Producer、Consumer、Queue、Exchange、Binding，逐个能说清职责（3.2）
理解消息可靠性	三环节防丢：confirm、两层持久化、ACK 时机；了解死信队列（3.4）
理解四大经典问题	消息丢失/重复消费/消息顺序/消息积压的成因与解法（3.5）
理解削峰填谷	瞬时洪峰先入队，消费者按能力匀速落库
完成点赞异步落库	Redis 先应答 → MQ → Consumer 写 MySQL
完成评论通知	评论 → MQ → 通知消费者
课堂实战	给“分享”做异步落库（第八节，课内启动、课下完成，闭环你的分享功能）

二、问题场景（先复现，再优化）

2.1 两难困境：性能解决了，数据还悬在半空

Day3 之后点赞只写 Redis，接口快了，但留下一个必须还的债：Redis 是内存存储，点赞关系必须最终落到 MySQL 才算真正安全（Day3 全链路预告里的“闭环”就差今天这一步）。而落库这件事怎么做都不对劲：

- 1

问题1：Redis 里的点赞数据必须最终持久化到 MySQL，怎么扛住瞬间5000条写入？
- 2

问题2：直接同步落库 → 又回到 Day1 的老路（4条SQL硬扛，RT 飙升）

2.2 三条落库路线的对比（为什么是 MQ）

路线	做法	问题
同步落库	接口里直接写 MySQL	回到 Day1 老路：洪峰直击数据库，RT 高
定时任务批量落库	每 5 秒把 Redis 增量扫进 MySQL	实现复杂（增量怎么标记？扫完怎么清？）；宕机窗口丢数据；实时性差
MQ 异步落库（本课）	点赞时顺手发一条消息，消费者按自己的节奏匀速写库	需要引入新中间件；需处理重复消费（Day4 幂等正好用上）

MQ 路线的本质是把“写数据库”从请求链路里剪下来，交给一个**缓冲带**暂存，再由专人慢慢消化——这个缓冲带就是队列。

2.3 答案：削峰填谷



水库的比喻：洪水（洪峰流量）先进水库（队列）蓄着，下游按闸门能力（消费速度）匀速放水（落库）。洪峰不再直接冲击河堤（数据库），这就是“削峰填谷”。

三、理论基础：消息队列原理与选型

3.1 MQ 是什么，三大作用分别解决什么问题

消息队列（Message Queue，MQ）本质是系统之间的一条**传送带**：发送方把“包裹”（消息）放上去就可以走人，接收方按自己的节奏取件处理。生活中的原型随处可见：餐厅后厨的订单夹（服务员不用站在灶台边等菜做好）、快递驿站（收件人不在家，包裹先存着）——它们都是“生产者不等消费者”的缓冲设计。

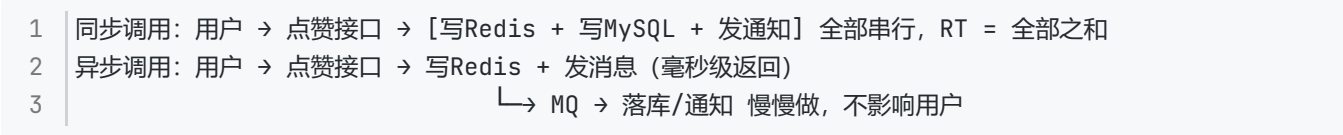
三大作用总览（背下来）：

作用	含义	本课案例
异步	耗时操作不阻塞主流程	评论通知不再阻塞评论接口
解耦	主流程不需要知道下游有谁	点赞接口不关心谁消费消息（落库/通知/...）
削峰	洪峰先入队，下游按能力消费	5000条/秒的点赞 → 匀速落库

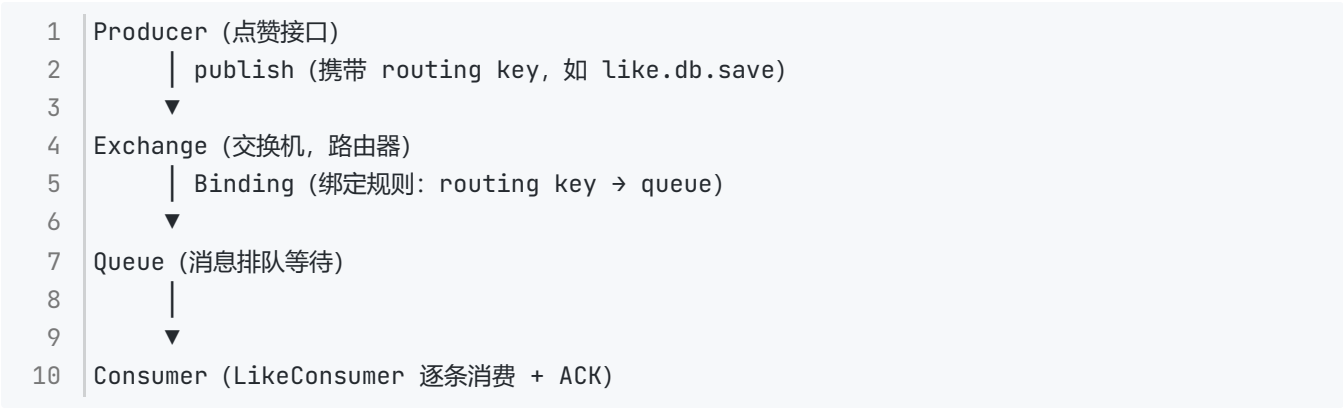
异步—算一笔 RT 账：假设评论接口要做三件事：写评论 20ms、发站内通知 30ms、推送短信 200ms。同步串行时用户要等 250ms；改异步后，写完评论发一条消息（约 5ms）就返回，用户只等 25ms——通知和短信在后台慢慢做，用户根本不需要等它们。**判断标准：哪些步骤用户不需要“当场拿到结果”，哪些就可以异步化。**

解耦—下游加人不用改上游：如果明天产品要求“点赞后给用户加积分”，同步架构下你得改点赞接口的代码、重新测试、重新上线；有了 MQ，只需新写一个积分消费者绑到同一个交换机上——点赞接口一行代码不改，甚至不知道积分消费者的存在。**主流程只负责“广播发生了什么”，谁关心谁自己订阅。**

削峰—队列是流量的蓄水池：MySQL 每秒只能安全写入约 1000 条，而热门事件能瞬间产生 5000 条/秒的点赞。没有队列，多出的 4000 条直接压垮数据库；有了队列，消息先堆着（Ready 积压），消费者按 500~1000 条/秒匀速处理，几秒后洪峰自然消化完。**代价是数据晚到几秒（软状态），而点赞正好是“最终一致可接受”的业务（Day3 一致性分级的伏笔在这里收回）。**



3.2 RabbitMQ 核心模型（每个角色都要能说清楚）



逐个角色说清楚：

- **Producer (生产者)**：发消息的一方。它只把消息交给 Exchange，**不直接关心消息最终进哪个队列**；
- **Exchange (交换机)**：路由器。收到消息后，拿消息的 routing key 去对照 Binding 规则，决定投给哪个（些）队列。**为什么要多这一层而不直接发队列？**——把“发消息”和“路由规则”解耦：以后要改投递规则（加队列、改匹配）只动 Binding，生产者和消费者代码都不用改；
- **Binding (绑定)**：交换机与队列之间的路由规则，声明“什么样的 routing key 进什么队列”；
- **Queue (队列)**：消息存储的地方，先进先出；管理台里的 Ready（待消费）/ Unacked（已投递未确认）两列就是它的健康指标；
- **Consumer (消费者)**：从队列取消息处理的一方，处理完要 ACK（确认）；
- **routing key 与通配符**：本课用 topic 交换机，routing key 是用 . 分段的路径（如 like.db.save）；Binding 里 * 匹配恰好一段，# 匹配零段或多段。本课代码绑定用的是 like.db.#，所以 like.db.save、like.db.cancel 甚至将来新增的 like.db.xxx.yyy 都能进同一个队列——扩展时不用改绑定。

四种 Exchange 类型（面试常考）：

类型	路由规则	典型用途举例
direct	routing key 完全相等才投递	精确分发：order.pay 只进支付队列
fanout	无视 routing key，广播给所有绑定队列	配置刷新通知：所有服务实例都收到同一条
topic	通配符匹配（* 一段 / # 多段）	本课：like.db.# 灵活扩展
headers	按消息头属性匹配（不看 routing key）	较少使用

面试常问：fanout 和 topic 都能广播，区别是什么？
(fanout 无条件全投，性能最好但不能挑；topic 可以按通配符精确地“选择性广播”，更灵活。)

3.3 主流 MQ 对比与选型理由

MQ	语言	单机吞吐	特点	适用场景
RabbitMQ (本课)	Erlang	万级	功能完整、管理台友好、路由灵活	中小规模业务消息
Kafka	Scala	百万级	日志式顺序存储、高吞吐	日志/大数据流
RocketMQ	Java	十万级	事务消息、阿里生态	电商交易类

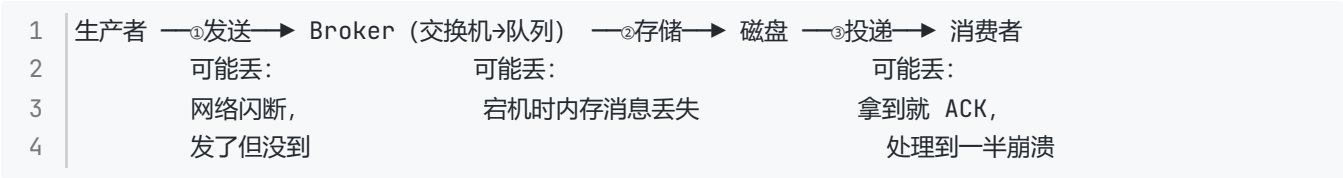
本课为什么选 RabbitMQ? 三个理由: ① 自带 Web 管理台, 队列积压、消费速率、连接状态一目了然, 对教学和中小项目极其友好; ② 路由模型灵活 (四种 Exchange), 业务消息场景表达力强; ③ 万级吞吐对实训项目的量级绰绰有余——选型不是选“最强的”, 而是选“够用的、团队驾驭得了的”。

一句话记住另外两位: Kafka 把消息当**日志**存 (顺序写磁盘 + 批量发送, 吞吐百万级), 适合日志采集、大数据管道, 但路由能力弱; RocketMQ 是阿里电商场景磨出来的, 招牌是**事务消息** (本地事务与发消息的原子性), 适合交易类业务。

面试常问: “为什么选 RabbitMQ 不选 Kafka?”——不要只背参数, 按“业务量级 + 功能需求 + 团队熟悉度”三层回答, 再补一句“日志流场景我会选 Kafka”。

3.4 消息可靠性：一条消息可能丢在哪三个环节（重点）

把一条消息从出生到落库的旅程画出来, 丢失风险点一目了然:



环节	丢失原因	保障手段	本课落地情况
① 生产端	发送失败未感知	publisher confirm 确认机制	未开启 (进阶, 见下文)
② Broker	服务器宕机	队列/消息持久化	✅ <code>new Queue(name, true)</code> 的 <code>true</code> 就是 <code>durable</code>
③ 消费端	拿到消息就 ACK, 处理到一半崩溃	手动/自动 ACK 的正确姿势	✅ Spring AMQP 默认 <code>AUTO</code> : 处理成功才确认

① **生产端: publisher confirm**。原理: 生产者发消息后不“发完就算”, Broker 真正把消息存入队列后会回一个 `confirm` (成功) 或 `nack` (失败); 生产者在回调里对失败消息重发或记日志告警。开启方式 (Spring Boot): `spring.rabbitmq.publisher-confirm-type: correlated + publisher-returns: true`。本课未开启——因为即使消息偶发丢失, Redis 里的点赞数据仍在, 可以用对账任务补偿 (见 3.5 消息丢失条目), 教学上先保证主线简单。

② **Broker: 持久化要两层都做**。队列持久化 (`durable=true`, 队列元信息落盘) + 消息持久化 (`deliveryMode=2`, 消息体落盘)。Spring AMQP 用 `Jackson2JsonMessageConverter` 发送时默认就是持久化消息, `new Queue(name, true)` 声明的就是持久化队列——两层本课都已具备。诚实说一句: 持久化不是 100% 保险——消息从内存刷到磁盘之间仍有一个极小的丢失窗口, 更强保障需要镜像队列/仲裁队列 (Quorum Queue, 多副本, 进阶了解)。

③ **消费端: ACK 的时机才是关键**。两种模式对比:

模式	行为	风险
自动 ACK (发完即确认)	Broker 投递出去就算消费成功	消费者处理到一半崩溃 → 消息永久丢失
处理完才 ACK (本课)	监听方法正常返回才确认; 抛异常则 <code>nack</code> 重新入队	不丢消息, 但“毒消息”会无限重投 → 需死信队列兜底

本课没有配 `acknowledge-mode: manual`, 用的是 Spring AMQP 默认的 `AUTO`——注意它的语义不是“收到就确认”, 而是**监听方法正常返回才确认、抛异常自动 nack 重投**, 对本课场景已经足够安全。手动 ACK (`Channel.basicAck/basicNack`) 适合需要精细控制时再用。

死信队列（DLX，进阶认知）：一条消息反复处理失败（如消息体非法、依赖服务长期不可用）会被无限重投，阻塞后面正常消息。解法：给队列配死信交换机，失败超过 N 次（或消息 TTL 到期、队列满）的消息被转入**死信队列**，不再重投；人工或定时任务事后处理。生产系统标配，本课不实现但要知道它存在。

3.5 用好 MQ 的四个经典问题（面试必考，提前认知）

问题一：消息丢失—三个环节各自设防（见 3.4）：生产端 confirm、Broker 持久化、消费端处理完才 ACK。还要补一道**对账兜底**：即使三层都做了，极端情况仍可能不一致，所以生产系统会跑定时对账任务（如每小时比对 **SCARD Like:{id}** 与表行数，不一致则补写）—Redis 里的数据在对账完成前都是安全的，这也是“先写 Redis”架构的额外好处。

问题二：重复消费—先接受一个事实：**重复投递无法从根上消除**（消费者处理成功但 ACK 前崩溃/网络超时，Broker 只能重投—它无法区分“没处理”和“处理了没来得及确认”）。所以方向不是“消灭重复”，而是**消费端幂等**：同一条消息处理 1 次和 N 次结果相同。Day4 的兵器库拿来用：本课 LikeConsumer 靠 **t_note_like** 的唯一索引捕获 DuplicateKeyException；也可用去重表 (msg_id 唯一键) 或 Redis SETNX 记已处理的消息 ID。

问题三：消息顺序—为什么会乱：多消费者并发取消息、重投把后来的消息插队、多队列并行消费。解法分三档：① 单队列单消费者（牺牲吞吐，简单可靠）；② 按业务 key 路由（如同一用户的消息进同一队列，Kafka 按 partition 天然支持）；③ **消息体携带状态，容忍乱序**—本课 LikeEvent 带 **liked=true/false** 布尔状态，消费者按消息内容覆盖式处理，而不是“收到就 +1”的增量式处理，乱序影响被压到最小（进阶方案：消息加版本号，旧版本直接丢弃）。

问题四：消息积压—Ready 持续上涨不回落，说明消费速度 < 生产速度。先判断原因：消费者挂了（重启/修复）还是单纯不够快？处理手段按优先级：① 扩容消费者（加实例、加 **concurrency** 并发数）；② 消费端改批量写入（攒 100 条一次 INSERT）；③ 临时降级非核心消费（如通知类暂停，保落库）；④ 生产端限流（Day8 的手段）。积压监控本身就是生产运维的日常：管理台看 Ready 曲线，告警阈值提前设好。

问题	一句话结论	本课的体现
消息丢失	三环节设防 + 对账兜底	持久化 + ACK (见 3.4)
重复消费	消灭不了重复，只能幂等	LikeConsumer 捕获唯一索引冲突 (Day4 呼应)
消息顺序	单队列/按key路由/携状态容忍乱序	liked 布尔覆盖式处理 (进阶话题)
消息积压	消费速度 < 生产速度，扩容/批量/限流	压测时观察 Ready 先升后降 (6.2 节)

3.6 动手：认识管理台（上课必做）

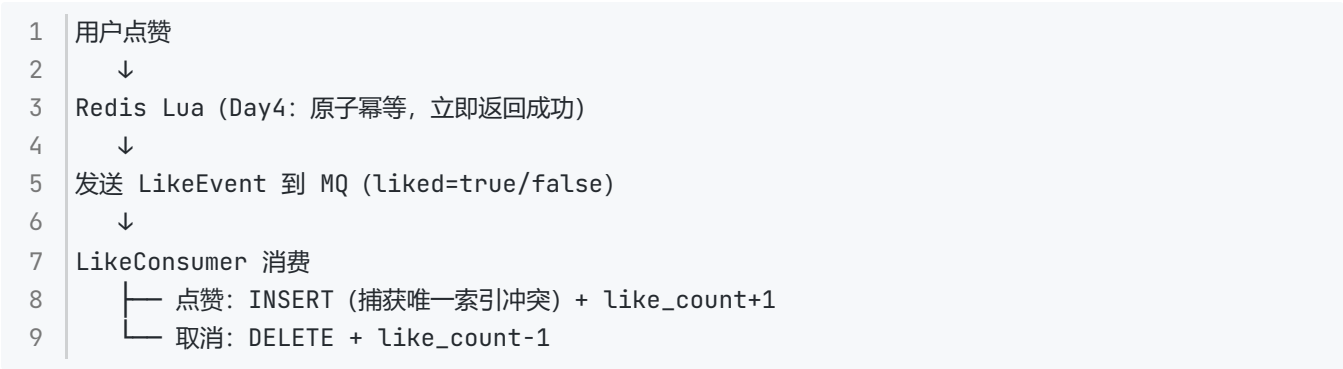
- 1. 打开 <http://localhost:15672> (guest/guest) ；
- 2. Exchanges 页: 查看 **xhs.exchange** , 展开看 3 条 Binding;
- 3. Queues 页: 查看 3 个队列, 理解 Ready (待消费) / Unacked (消费中) 两列;
- 4. 点开 **like.db.queue** → Publish message, 手动发一条测试消息, 观察消费日志。

四、方案设计

4.1 拓扑设计 (Topic 交换机)

1	Exchange: xhs.exchange (topic)	
2	├─ like.db.# → like.db.queue	点赞落库
3	├─ comment.notify.# → comment.notify.queue	评论通知
4	└─ note.es.# → note.es.queue	笔记同步ES (Day8使用)

4.2 点赞异步落库流程



4.3 消息体设计

消息	字段	说明
LikeEvent	noteId, userId, liked	liked=true 点赞 / false 取消
CommentEvent	noteId, authorId, userId, content	用于通知笔记作者

五、编码实现

按 [code/day6/](#) 目录完成：

序号	文件	操作
1	pom.xml	【替换】新增 spring-boot-starter-amqp
2	application.yml	【替换】新增 rabbitmq 配置
3	config/RabbitConfig.java	【新增】交换机/队列/绑定 + JSON消息转换器
4	dto/LikeEvent.java	【新增】点赞消息体
5	dto/CommentEvent.java	【新增】评论消息体
6	service/InteractService.java	【替换】点赞成功后发送 LikeEvent
7	service/CommentService.java	【替换】评论成功后发送 CommentEvent
8	consumer/LikeConsumer.java	【新增】点赞落库消费者
9	consumer/NotificationConsumer.java	【新增】评论通知消费者

六、验证与压测

6.1 功能验证

- 1. 打开管理台 <http://localhost:15672> (guest/guest) , 确认 3 个队列存在;
- 2. 前端点赞一次：
 - Redis 立即更新 (体验不变) ；
 - 稍等 1~2 秒后查库: `SELECT * FROM t_note_like ORDER BY id DESC LIMIT 1` 出现新记

录；

3. 前端评论一次：观察后端控制台打印的通知日志；
4. 取消点赞一次：确认数据库记录被删除、计数减 1。

6.2 削峰验证（核心验收）

1. JMeter 压测点赞：并发 500，持续 30 秒；
2. 压测进行中刷新管理台，观察 `like.db.queue` 的 Ready 积压数量先升后降；
3. 压测结束后等待队列清零，执行核对：

```
1 Redis: SCARD like:1      = MySQL: SELECT COUNT(*) FROM t_note_like WHERE note_id=1
2 Redis: GET like:count:1  = MySQL: SELECT like_count FROM t_note WHERE id=1
```

两边一致 = 最终一致性达成。

七、结果记录表

指标	优化前 (Day1基线)	优化后	变化
点赞接口平均RT			
点赞接口P95			
MySQL 瞬时写入峰值	5000/秒级	匀速	
队列最大积压	-		
数据核对结果	-	一致/不一致	

八、课堂实战：让“分享”落库（自己动手）

你从 Day3 开始的“分享”功能至今只活在 Redis 里 — 今天让它落库，形成完整闭环：
Redis 先应答 (Day3) → Lua 幂等 (Day4) → MQ 异步落库 (今天)。

需求：分享成功后发送 `ShareEvent` 到 MQ，由新的 `ShareConsumer` 写入 `t_note_share` 表：

- routing key: `share.db.save` (挂在已有的 `xhs.exchange` 上, 和 `like.db.*` 平行)；
- 消费者幂等：捕获唯一索引冲突 (Day4 学的招)；
- 取消分享：发 `shared = false`，消费者执行 DELETE。

实现步骤：

1. 建表 (执行一次)：

```
1 CREATE TABLE t_note_share (
2     id BIGINT PRIMARY KEY AUTO_INCREMENT,
3     note_id BIGINT NOT NULL,
4     user_id BIGINT NOT NULL,
5     create_time DATETIME DEFAULT CURRENT_TIMESTAMP,
6     UNIQUE KEY uk_user_note (user_id, note_id)
7 );
```

2. 新建 `dto/ShareEvent.java` (仿 `LikeEvent`: `noteId`、`userId`、`shared`) ;
3. `RabbitConfig.java` 声明 `share.db.queue` 并绑定 `share.db.*` ;
4. 新建 `consumer/ShareConsumer.java` , 仿照 `LikeConsumer` : 点赞时 `INSERT` (捕获重复键异常) , 取消时 `DELETE` ;
5. 修改你 Day4 写的 `share` 方法: Redis 操作成功后 `rabbitTemplate.convertAndSend(...)` 发消息。

关键代码提示 (消费者骨架) :

```
1 @RabbitListener(queues = "share.db.queue")
2 public void onMessage(ShareEvent event) {
3     if (event.isShared()) {
4         try {
5             shareMapper.insert(new NoteShare(event.getNoteId(), event.getUserId()));
6         } catch (DuplicateKeyException e) {
7             // 重复消费 / 并发重复: 唯一索引兜底, 直接忽略 (幂等)
8         }
9     } else {
10        shareMapper.deleteByUserAndNote(event.getUserId(), event.getNoteId());
11    }
12 }
```

验收标准:

分享一次: Redis 立即更新, 1~2 秒后 `SELECT * FROM t_note_share` 出现记录;

管理台能看到 `share.db.queue` 及在线消费者;

500 并发压分享接口: 队列积压先升后降, 结束后 `SCARD share:{id}` 与表行数一致;

取消分享后表记录被删除。

选做挑战: 如果消费失败 (比如数据库临时不可用) , 怎么让消息不丢? (提示: 手动 `ACK` + 失败重试, 查一下 `@RabbitListener` 的 `ackMode`)

九、思考题

1. 如果消费者处理到一半服务重启了, 消息会丢吗? (提示: 手动`ACK`、持久化)
2. 为什么落库消费者要捕获唯一索引冲突而不是先查再插?
3. 积压越来越多说明什么? 应该怎么处理? (提示: 扩容消费者、限流)